

* Modules in Python :-

Index

- => Purpose of modules
- => Definition of module
- => Types of modules
 - a) pre-defined modules
 - b) programmer/user/ custom defined modules
- => Steps for developing programmer defined module
- => Number of Approaches to re-use module.
 - a) By using 'import' statement
 - b) By using 'from', 'import' statement
- => programming examples
- => Reloading modules
- => 'imp'/'importlib' module
- => programming Example
- => Use Cases.

* Modules in Python :-

→ we know that functions are used for "performing certain operations and provides code Reusability within the same program but not able to provide Code Reusability across the programs."

→ The purpose of modules concept is that "to Re-use the functions, global variables and Class Names" from one program to another program provided Both the programs present in same folder.

=> Definition of module:-

→ A module is a collection of functions, global variable names and class names.

Types of modules:-

→ In python programming, we have two types of modules. they are

① pre-Defined modules

② programmer OR user OR custom defined
value

① pre-defined modules:-

→ These modules are already defined by python lang developers and available in python software and used all python lang programmers and for dealing with universal requirements.

Examples: functools, sys, calendar, re, pickle, threading, csv... etc.

→ out of many pre-defined modules, by default one of the pre-defined module called "builtins" imported to all python programs as it is called default imported python module.

② programmer OR user OR custom defined module:-

→ These modules are developed by python programmer and available in python project and used by other members of same project for dealing with common requirements.

Example: aop, mathinfo, icici

* Development of programmer-defined module:-

→ To develop programmer-defined module, we must use the following steps:-

step ①:- Define variables (Global variables)

step ②:- Define functions

step ③:- Define classes

⇒ After developing step-①, step②, step③, we must save on some file name with an extension .py (filename.py) and it is treated as module name.

→ Hence every python file is treated as module name.

→ When file name treated as a module name, internally python execution environment creates a folder automatically on the name of `--pycache--` and it contains module name on the name of `"filename.cpython-311.pyc"`.

Example:

```
--pycache-- <--- Folder name
--
aop.cpython-311.pyc <--- module name
mathsinfo.cpython-311.pyc <--- module Name
icici.cpython-311.pyc <--- module Name
```

Example

#mathsinfo --- Filename acts as module name

```
① PI = 3.14
   E = 2.71 # Here PI and E are called Global variables
print("val of PI =", PI)
print("val of E =", E)
```

```
② #SE1.py
# Here SE1 is trying to re-use the global variables of one
  program in another program.
```

```
import mathsinfo
print("val of PI =", mathsinfo.PI)
print("val of E =", mathsinfo.E).
```

==

ex(2). #aop.py

```
def sumop(a,b):  
    print("sum({},{})={}".format(a,b,a+b))  
  
def subop(a,b):  
    print("sub({},{})={}".format(a,b,a-b))  
  
def mulop(a,b):  
    print("mul({},{})={}".format(a,b,a*b))
```

→ #SE2.PY

pyCalc - dunder
mgre

```
import aop  
aop.sumop(10,20)  
aop.subop(10,20)  
aop.mulop(10,20)
```

ex(3).

#icici.py — — — File name & module name

bname = "ICICI"

addr = "HYD" # Here bname & addr are global vars

def simpleint(): # function definition.

p = float(input("Enter principle amount:"))

t = float(input("Enter time:"))

r = float(input("Enter rate of interest:"))

Cal si

si = (p*t*r)/100.

totalamt = p + si

print("x" * 50)

(print("Simple interest calculation"))

print("\t Principle Amount: {}".format(p))

+
r
si.

total

#SE3.PY

import icici

print("Bank Name: {}".format(icici.bname))

print("Bank Address: {}".format(icici.addr))

icici.simpleint()